

linear-regression

September 19, 2026

1 Linear Regression

In Machine Learning and this notebook we use Scikit-learn a lot.

1.0.1 What is scikit-learn used for?

Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python. It provides a selection of efficient tools for machine learning and statistical modeling including classification, regression, clustering and dimensionality reduction via a consistence interface in Python.

What is linear regression used for? Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable you want to predict is called the dependent variable. The variable you are using to predict the other variable's value is called the independent variable.

2 Making Predictions with Linear Regression

Given the representation is a linear equation, making predictions is as simple as solving the equation for a specific set of inputs.

Let's make this concrete with an example. Imagine we are predicting weight (y) from height (x). Our linear regression model representation for this problem would be:

$$y = B0 + B1 * x1$$

or

$$\text{weight} = B0 + B1 * \text{height}$$

Where B0 is the bias coefficient and B1 is the coefficient for the height column. We use a learning technique to find a good set of coefficient values. Once found, we can plug in different height values to predict the weight.

For example, lets use $B0 = 0.1$ and $B1 = 0.5$. Let's plug them in and calculate the weight (in kilograms) for a person with the height of 182 centimeters.

$$\text{weight} = 0.1 + 0.5 * 182$$

$$\text{weight} = 91.1$$

You can see that the above equation could be plotted as a line in two-dimensions. The B_0 is our starting point regardless of what height we have. We can run through a bunch of heights from 100 to 250 centimeters and plug them to the equation and get weight values, creating our line.

Now that we know how to make predictions given a learned linear regression model, let's look at some rules of thumb for preparing our data to make the most of this type of model.

3 Import & Install Libraries

```
[1]: # !pip install hvplot
```

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import hvplot.pandas

from sklearn.model_selection import train_test_split

from sklearn import metrics

from sklearn.linear_model import LinearRegression

%matplotlib inline
```

3.1 Check out the Data

```
[3]: df=pd.read_csv('./Real estate.csv')
```

```
[4]: df.head()
```

```
[4]:
```

	No	X1 transaction date	X2 house age	\
0	1	2012.917	32.0	
1	2	2012.917	19.5	
2	3	2013.583	13.3	
3	4	2013.500	13.3	
4	5	2012.833	5.0	

	X3 distance to the nearest MRT station	X4 number of convenience stores	\
0	84.87882	10	
1	306.59470	9	
2	561.98450	5	
3	561.98450	5	
4	390.56840	5	

	X5 latitude	X6 longitude	Y house price of unit area
0	24.98298	121.54024	37.9

1	24.98034	121.53951	42.2
2	24.98746	121.54391	47.3
3	24.98746	121.54391	54.8
4	24.97937	121.54245	43.1

```
[5]: print(df.shape)
```

```
(414, 8)
```

```
[6]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 414 entries, 0 to 413
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0    No                                     414 non-null    int64
1    X1 transaction date                  414 non-null    float64
2    X2 house age                        414 non-null    float64
3    X3 distance to the nearest MRT station 414 non-null    float64
4    X4 number of convenience stores       414 non-null    int64
5    X5 latitude                         414 non-null    float64
6    X6 longitude                        414 non-null    float64
7    Y house price of unit area           414 non-null    float64
dtypes: float64(6), int64(2)
memory usage: 26.0 KB
```

```
[7]: df.corr()
```

```
[7]:
```

	No	X1 transaction date \
No	1.000000	-0.048658
X1 transaction date	-0.048658	1.000000
X2 house age	-0.032808	0.017549
X3 distance to the nearest MRT station	-0.013573	0.060880
X4 number of convenience stores	-0.012699	0.009635
X5 latitude	-0.010110	0.035058
X6 longitude	-0.011059	-0.041082
Y house price of unit area	-0.028587	0.087491

	X2 house age \
No	-0.032808
X1 transaction date	0.017549
X2 house age	1.000000
X3 distance to the nearest MRT station	0.025622
X4 number of convenience stores	0.049593
X5 latitude	0.054420
X6 longitude	-0.048520
Y house price of unit area	-0.210567

	X3 distance to the nearest MRT station
\	
No	-0.013573
X1 transaction date	0.060880
X2 house age	0.025622
X3 distance to the nearest MRT station	1.000000
X4 number of convenience stores	-0.602519
X5 latitude	-0.591067
X6 longitude	-0.806317
Y house price of unit area	-0.673613

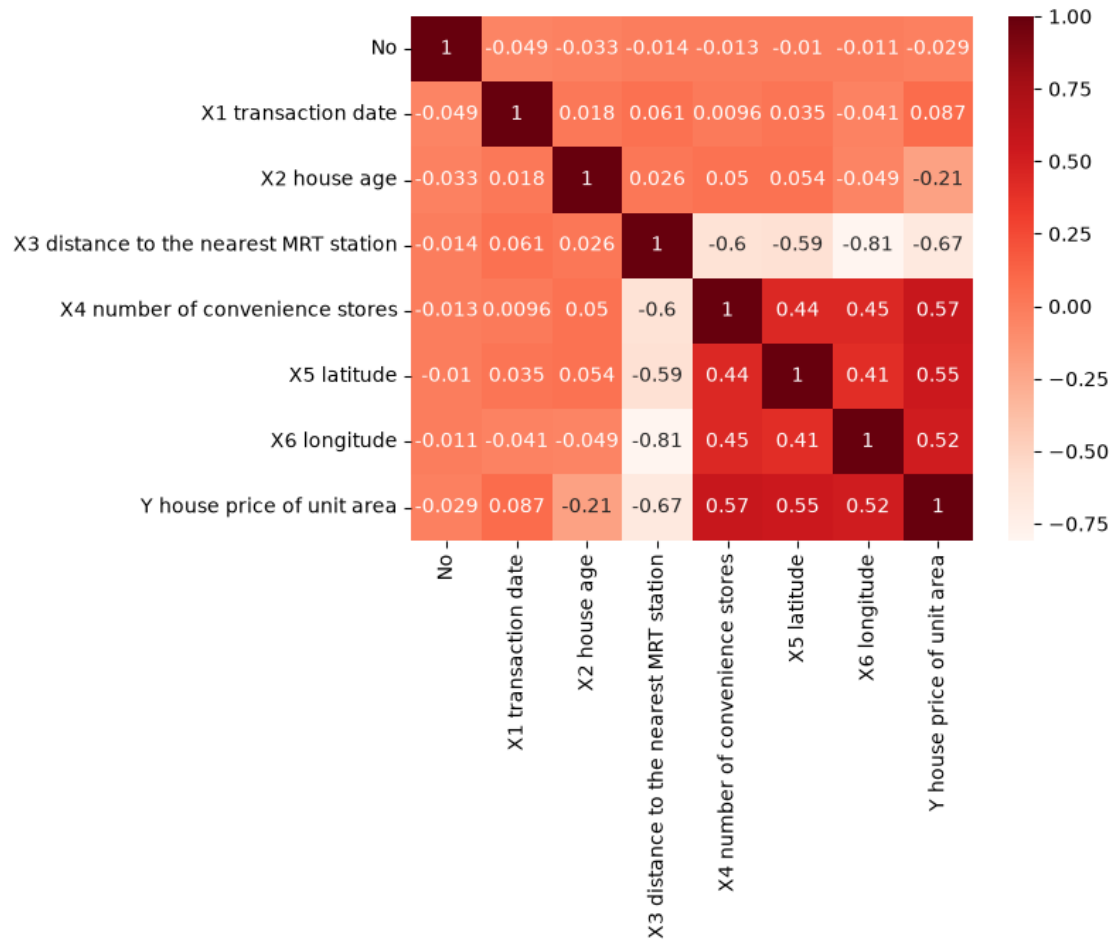
	X4 number of convenience stores \
No	-0.012699
X1 transaction date	0.009635
X2 house age	0.049593
X3 distance to the nearest MRT station	-0.602519
X4 number of convenience stores	1.000000
X5 latitude	0.444143
X6 longitude	0.449099
Y house price of unit area	0.571005

	X5 latitude	X6 longitude \
No	-0.010110	-0.011059
X1 transaction date	0.035058	-0.041082
X2 house age	0.054420	-0.048520
X3 distance to the nearest MRT station	-0.591067	-0.806317
X4 number of convenience stores	0.444143	0.449099
X5 latitude	1.000000	0.412924
X6 longitude	0.412924	1.000000
Y house price of unit area	0.546307	0.523287

	Y house price of unit area
No	-0.028587
X1 transaction date	0.087491
X2 house age	-0.210567
X3 distance to the nearest MRT station	-0.673613
X4 number of convenience stores	0.571005
X5 latitude	0.546307
X6 longitude	0.523287
Y house price of unit area	1.000000

```
[8]: sns.heatmap(df.corr(), annot=True, cmap='Reds')
```

```
[8]: <Axes: >
```



4 Exploratory Data Analysis (EDA)

```
[9]: # sns.pairplot(df)
```

5 Training a Linear Regression Model

5.1 X and y arrays

```
[10]: # X=df.drop('Y house price of unit area', axis=1)

# y=df['X4 number of convenience stores']

X = df.drop(columns=['Y house price of unit area', 'No', "X1 transaction date",
↪ "X2 house age"])
```

```
y = df['Y house price of unit area']
```

```
[11]: print("X=",X.shape,"\ny=", y.shape)
```

```
X= (414, 4)
```

```
y= (414,)
```

5.2 Train Test Split

Now let's split the data into a training set and a testing set. We will train our model on the training set and then use the test set to evaluate the model.

```
[12]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  
↳ random_state=101)
```

```
[13]: X_train.shape
```

```
[13]: (289, 4)
```

```
[14]: X_test.shape
```

```
[14]: (125, 4)
```

6 Linear Regression

```
[15]: model = LinearRegression()
```

```
[16]: model.fit(X_train, y_train)
```

```
[16]: LinearRegression()
```

6.1 Model Evaluation

```
[17]: model.coef_
```

```
[17]: array([-4.13410211e-03,  1.17953550e+00,  2.35911752e+02,  4.71579161e+01])
```

```
[18]: pd.DataFrame(model.coef_, X.columns, columns=['Coefficients'])
```

```
[18]:
```

	Coefficients
X3 distance to the nearest MRT station	-0.004134
X4 number of convenience stores	1.179535
X5 latitude	235.911752
X6 longitude	47.157916

6.2 Predictions from our Model

```
[19]: y_pred = model.predict(X_test)
```

6.3 Regression Evaluation Metrics

Here are three common evaluation metrics for regression problems:

- **Mean Absolute Error (MAE)** is the mean of the absolute value of the errors:

$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Mean Squared Error (MSE)** is the mean of the squared errors:

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Root Mean Squared Error (RMSE)** is the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Comparing these metrics: - **MAE** is the easiest to understand, because it's the average error. - **MSE** is more popular than MAE, because MSE “punishes” larger errors, which tends to be useful in the real world. - **RMSE** is even more popular than MSE, because RMSE is interpretable in the “y” units.

All of these are **loss functions**, because we want to minimize them.

```
[20]: MAE= metrics.mean_absolute_error(y_test, y_pred)
      MSE=metrics.mean_squared_error(y_test, y_pred)
      RMSE= np.sqrt(MSE)
```

```
[21]: MAE
```

```
[21]: 6.092476998635309
```

```
[22]: MSE
```

```
[22]: 57.1719352844051
```

```
[23]: RMSE
```

```
[23]: np.float64(7.561212553843801)
```

```
[24]: df['X4 number of convenience stores'].mean()
```

```
[24]: np.float64(4.094202898550725)
```

6.4 Residual Histogram

- Often for Linear Regression it is a good idea to separately evaluate residuals

$$(y - \hat{y})$$

and not just calculate performance metrics (e.g. RMSE).

- Let's explore why this is important...
- The residual errors should be random and close to a normal distribution.

```
[25]: test_residual= y_test - y_pred
```

```
print(test_residual)
```

```
176      6.585609
```

```
347      4.325558
```

```
307      1.153119
```

```
299     -6.672654
```

```
391      3.164413
```

```
...
```

```
16      23.589240
```

```
31     -21.492475
```

```
36      -7.485957
```

```
41       4.439754
```

```
24      -3.383089
```

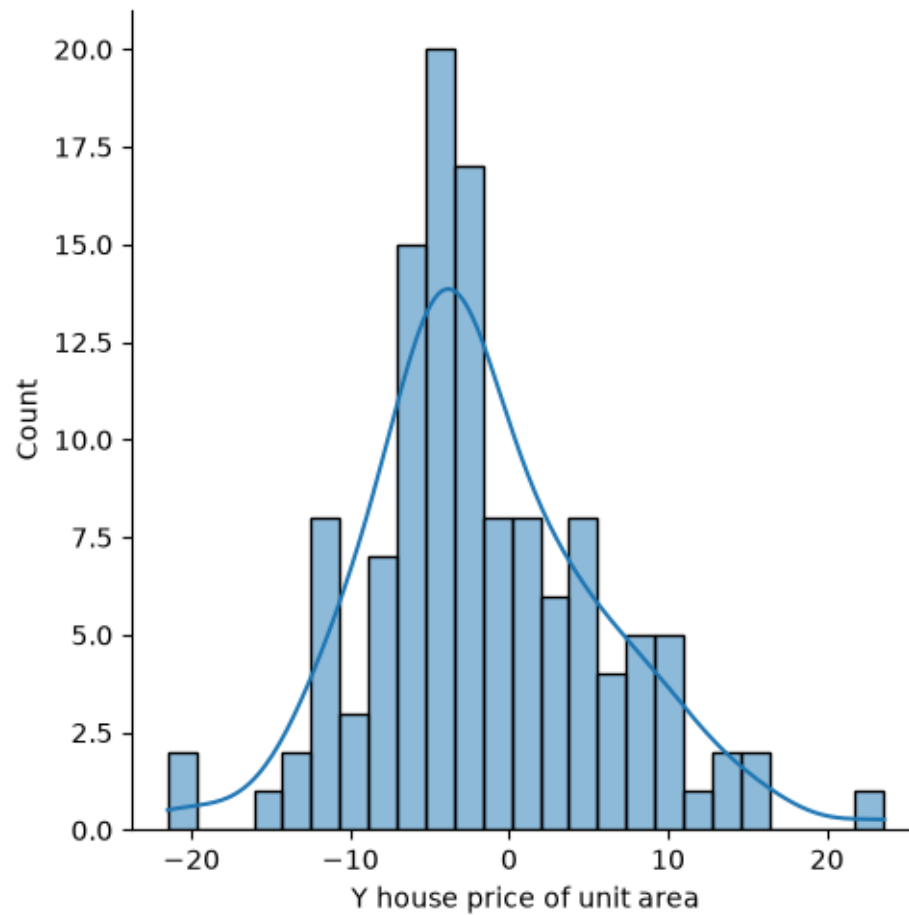
```
Name: Y house price of unit area, Length: 125, dtype: float64
```

```
[26]: pd.DataFrame({'Error Values': (test_residual)}).hvplot.kde()
```

```
[26]: :Distribution      [Error Values]      (Density)
```

```
[27]: sns.displot(test_residual, bins=25, kde=True)
```

```
[27]: <seaborn.axisgrid.FacetGrid at 0x1e638609400>
```

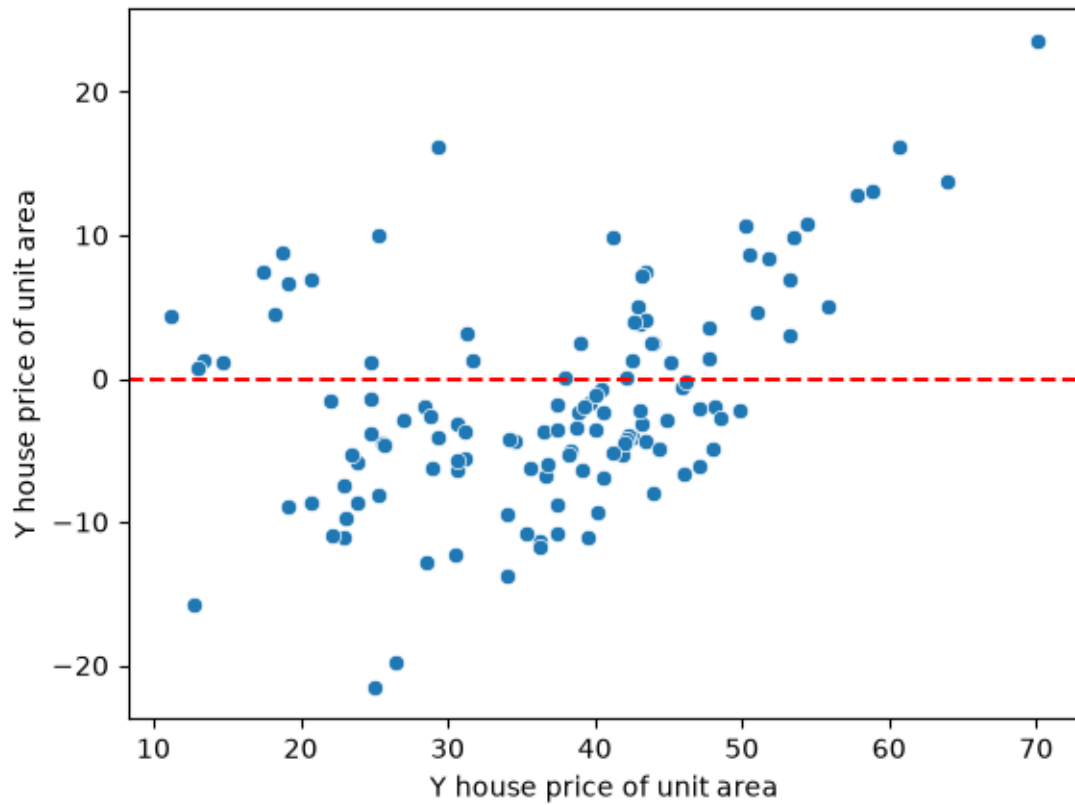


- Residual plot shows residual error VS. true y value.

```
[28]: sns.scatterplot(x=y_test, y=test_residual)

plt.axhline(y=0, color='r', ls='--')
```

```
[28]: <matplotlib.lines.Line2D at 0x1e63aa76660>
```



- Residualplot showing a clear pattern, indicating Linear Regression no valid!

7 Finished, but you can copy this notebook and start practicing.